

**LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 4
“PROSEDUR”**



**DISUSUN OLEH:
NADIFA AZKHIA SYARIF
109082530002
S1 IF-13-02**

**DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

DASAR TEORI

Dalam pemrograman, prosedur merupakan salah satu bentuk subprogram yang digunakan untuk mengelompokkan sekumpulan instruksi menjadi satu kesatuan logis yang dapat dipanggil berulang kali dalam suatu program. Penggunaan prosedur bertujuan untuk mengurangi kompleksitas program, meningkatkan keterbacaan, serta mendukung konsep modularitas dalam pengembangan perangkat lunak. Menurut modul praktikum, prosedur tidak memiliki nilai kembalian dan tidak menggunakan pernyataan *return*, melainkan menghasilkan efek langsung terhadap jalannya program seperti menampilkan output atau memodifikasi data tertentu (Modul Praktikum Algoritma dan Pemrograman 2, 2024).

Secara umum, prosedur dideklarasikan di luar program utama dan dapat dipanggil kapan saja sesuai kebutuhan. Struktur prosedur terdiri dari nama prosedur, parameter (jika ada), serta badan prosedur yang berisi instruksi-instruksi yang akan dijalankan. Dengan adanya prosedur, program besar dapat dipecah menjadi bagian-bagian kecil yang lebih terstruktur sehingga lebih mudah dipahami dan dikelola.

Pemanggilan prosedur dilakukan dengan menuliskan nama prosedur beserta argumen yang sesuai dengan parameter yang telah dideklarasikan. Argumen yang diberikan harus memiliki jumlah, tipe data, dan urutan yang sama dengan parameter formal pada prosedur. Hal ini penting agar tidak terjadi kesalahan saat eksekusi program.

Parameter dalam prosedur berfungsi sebagai media komunikasi antara program utama dengan subprogram. Parameter dibedakan menjadi dua, yaitu parameter formal dan parameter aktual. Parameter formal adalah parameter yang dituliskan pada saat deklarasi prosedur, sedangkan parameter aktual adalah nilai atau variabel yang diberikan saat prosedur dipanggil. Konsep ini sejalan dengan prinsip pemrograman terstruktur yang dijelaskan oleh Brian W. Kernighan dan Dennis M. Ritchie dalam pengembangan bahasa pemrograman modern.

Selain itu, terdapat dua mekanisme pengiriman parameter, yaitu *pass by value* dan *pass by reference*. Pada *pass by value*, nilai parameter aktual disalin ke parameter formal sehingga perubahan yang terjadi di dalam prosedur tidak memengaruhi nilai variabel asli. Sedangkan pada *pass by reference*, alamat memori dari parameter aktual dikirimkan ke parameter formal, sehingga perubahan yang terjadi di dalam prosedur akan memengaruhi nilai variabel asli. Pendekatan ini banyak digunakan untuk meningkatkan efisiensi, terutama dalam pengolahan data berukuran besar (The Go Authors, n.d.).

Prosedur memiliki perbedaan mendasar dengan fungsi. Jika fungsi digunakan untuk menghasilkan nilai kembalian, maka prosedur lebih difokuskan pada pelaksanaan aksi atau proses tertentu tanpa mengembalikan nilai. Konsep modular seperti ini merupakan salah satu dasar penting dalam pengembangan perangkat lunak yang efisien dan terstruktur, sebagaimana dijelaskan oleh Donald E. Knuth dalam kajiannya mengenai algoritma dan struktur program.

Dengan demikian, penggunaan prosedur dalam pemrograman sangat penting untuk meningkatkan efisiensi, keterbacaan, serta kemudahan dalam pengembangan dan pemeliharaan program.

CONTOH SOAL

1. Latihan1

Source Code:

```
package main

import "fmt"

func main() {
    var bilangan int
    var pesan string

    fmt.Scan(&bilangan, &pesan)
    cetakPesan(pesan, bilangan)
}

func cetakPesan(M string, flag int) {
    var jenis string = ""

    if flag == 0 {
        jenis = "error"
    } else if flag == 1 {
        jenis = "warning"
    } else if flag == 2 {
        jenis = "informasi"
    }

    fmt.Println(M, jenis)
}
```

Output:

2

2

2 informasi

Deskripsi Program:

Program ini membaca sebuah bilangan yang merepresentasikan jenis pesan dan sebuah teks pesan, kemudian mencetak pesan beserta jenisnya. Jenis pesan ditentukan dari kode bilangan: kode tertentu mewakili error, warning, atau informasi. Program menggunakan prosedur modular untuk memproses kode dan menentukan jenis pesan secara otomatis. Pendekatan ini membuat program mudah diperluas dan diterapkan pada sistem notifikasi atau logging sederhana, sehingga setiap pesan dapat disajikan secara konsisten sesuai kategorinya.

SOAL LATIHAN

Statement procedur

1. Source Code:

```
package main

import (
    "fmt"
)

func factorial(n int) int {
    result := 1
    for i := 2; i <= n; i++ {
        result *= i
    }
    return result
}

func permutation(n, r int) int {
    return factorial(n) / factorial(n-r)
}

func combination(n, r int) int {
    return factorial(n) / (factorial(r) * factorial(n-r))
}

func main() {
    var a, b, c, d int
    fmt.Scan(&a, &b, &c, &d) // input a b c d

    P1 := permutation(a, c)
    C1 := combination(a, c)
    fmt.Println(P1, C1)

    P2 := permutation(b, d)
    C2 := combination(b, d)
    fmt.Println(P2, C2)
}
```

Output:

5 10 3 10

60 10

3628800 1

Deskripsi Program:

Program ini dibuat untuk membantu mahasiswa menghitung permutasi dan kombinasi dari sepasang bilangan. Program ini menerima empat bilangan sebagai masukan, di mana setiap pasangan bilangan dihitung permutasi dan kombinasinya menggunakan rumus matematika yang sesuai. Pendekatan modular digunakan dengan prosedur untuk menghitung faktorial, permutasi, dan kombinasi secara terpisah, sehingga perhitungan menjadi terstruktur dan mudah diikuti. Program ini berguna untuk memahami konsep kombinasi dan permutasi secara langsung melalui implementasi kode, sekaligus melatih kemampuan pemrograman modular.

2. Source Code:

```
package main

import (
    "fmt"
    "strings"
)

func hitungSkor(waktu []int, soal *int, skor *int) {
    *soal = 0
    *skor = 0
    for _, t := range waktu {
        if t <= 300 {
            *soal++
            *skor += t
        }
    }
}

func main() {
    var nama string
    var waktu [8]int

    pemenang := ""
    maxSoal := -1
    minSkor := 0

    for {
        fmt.Scan(&nama)
        if strings.ToLower(nama) == "selesai" {
            break
        }

        for i := 0; i < 8; i++ {
            fmt.Scan(&waktu[i])
        }

        var soal, skor int
        hitungSkor(waktu[:], &soal, &skor)
```

```

        if soal > maxSoal || (soal == maxSoal && skor <
minSkor) {
            pemenang = nama
            maxSoal = soal
            minSkor = skor
        }
    }

    fmt.Println(pemenang, maxSoal, minSkor)
}

```

Output:

Astuti 20 50 301 301 61 71 75 10

Bertha 25 47 301 26 50 60 65 21

Selesai

Bertha 7 294

Deskripsi Program:

Program ini dirancang untuk menentukan pemenang kompetisi berdasarkan jumlah soal yang diselesaikan dan total waktu pengerjaan. Setiap peserta memiliki daftar waktu pengerjaan untuk beberapa soal, dan jika sebuah soal tidak diselesaikan atau tidak dikirim, dianggap selesai dalam waktu maksimal. Program menggunakan prosedur modular untuk menghitung skor setiap peserta dengan menghitung jumlah soal yang berhasil dan total waktu yang diperlukan. Pemenang ditentukan pertama-tama oleh jumlah soal terbanyak, dan jika sama, ditentukan oleh waktu total terkecil. Program ini membantu menilai peserta secara adil berdasarkan kriteria yang jelas.

3. Source Code:

```

package main

import "fmt"

func cetakDeret(n int) {
    for n != 1 {
        fmt.Print(n, " ")
        if n%2 == 0 {
            n = n / 2
        } else {
            n = 3*n + 1
        }
    }
    fmt.Println(1) // suku terakhir selalu 1
}

```

```
}  
  
func main() {  
    var n int  
    fmt.Scan(&n)  
    cetakDeret(n)  
}
```

Output:

22

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

Deskripsi Program:

Program ini mencetak deret bilangan yang dimulai dari sebuah bilangan positif dan mengikuti aturan tertentu: jika bilangan genap, suku berikutnya setengah dari bilangan saat itu; jika ganjil, suku berikutnya tiga kali bilangan ditambah satu. Deret berakhir ketika mencapai nilai satu. Prosedur modular digunakan untuk mencetak seluruh deret, sehingga setiap suku dicetak secara terstruktur. Program ini membantu mahasiswa memahami konsep deret bilangan, iterasi, dan rekursi sederhana melalui implementasi algoritma yang jelas dan modular.

DAFTAR PUSTAKA

Modul Praktikum Algoritma dan Pemrograman 2. 2024. *Modul 4: Prosedur*.

Donald E. Knuth. 1997. *The Art of Computer Programming*. Boston: Addison-Wesley.

Brian W. Kernighan, dan Dennis M. Ritchie. 1988. *The C Programming Language*. New Jersey: Prentice Hall.

The Go Authors. n.d. *The Go Programming Language Specification*. Diakses dari:
<https://golang.org>